

DeltaCache: Layer-Aware KV Cache Management for Efficient LLM Inference

Anonymous Authors

Abstract—Transformer layers exhibit significant *reuse heterogeneity* during inference: late layers encode task-specific semantic features with high reuse value, while early layers capture local patterns less critical for cache retention. We present DELTACACHE, a layer-aware KV cache management system that prioritizes late-layer cache entries during eviction. Combined with attention-based importance refinement and efficient prefix matching, DELTACACHE achieves $5.06\times$ average speedup on Mistral-7B with numerical equivalence at fp16 precision (max difference <0.016). On long-prefix workloads, we observe up to $10.90\times$ speedup with 17–22% memory overhead. Under memory pressure, importance-aware eviction achieves 66% higher hit rate than LRU. We identify practical deployment conditions and limitations, providing clear guidelines for practitioners.

Index Terms—Large Language Models, KV Cache, Layer-Aware Caching, Transformer Inference, Neural Network Optimization

I. INTRODUCTION

The Key-Value (KV) cache is a critical component in transformer inference, storing computed attention states to enable efficient autoregressive generation [1]. As Large Language Models (LLMs) scale to billions of parameters and longer context windows, KV cache management has become a significant bottleneck affecting both memory consumption and computational efficiency [2]. For a model with L layers, hidden dimension d , and sequence length n , the KV cache requires $O(2Lnd)$ memory, which can exceed tens of gigabytes for long-context scenarios.

Existing KV cache optimization methods typically treat all cached entries uniformly, applying the same eviction policies (e.g., LRU, LFU) regardless of which transformer layer produced them [3], [4]. This uniform treatment overlooks a fundamental characteristic of transformer architectures: *different layers serve different computational purposes*.

Our key observation. Transformer layers exhibit significant *reuse heterogeneity*—the value of caching KV states varies systematically across layers. This arises from functional specialization [5], [6]: early layers (0–30%) capture local syntactic patterns, while late layers (70–100%) encode task-specific semantic features that directly influence predictions [7].

This has a practical implication for cache management: when eviction becomes necessary, not all entries are equally valuable. Late-layer KV states encode features critical for prediction quality and should receive higher retention priority than early-layer entries capturing more generic patterns.

Our approach. Motivated by this observation, we present DELTACACHE, a KV cache management system that explicitly exploits layer-wise reuse heterogeneity. Our core mechanism

assigns layer-dependent importance weights using a sigmoid function that models the transition from syntactic to semantic processing:

$$w_l = \frac{1}{1 + e^{-k(l/L - \tau)}} \quad (1)$$

where l is the layer index, L is total layers, k controls transition steepness, and τ is the transition point. This gives late layers approximately $4.9\times$ higher retention priority than early layers. When memory pressure forces eviction, early-layer entries are evicted first, preserving the semantically important late-layer representations.

Contributions. Our main contribution is **layer-aware KV cache prioritization** that exploits reuse heterogeneity across transformer layers. To support this core mechanism, we implement:

- Attention-based importance refinement that uses attention scores to further distinguish entries within the same layer
- A prefix tree structure for efficient cache lookup and sharing across requests with $O(n)$ complexity
- Hierarchical GPU/CPU memory management with proactive offloading and predictive prefetching

We conduct comprehensive experiments on Mistral-7B and TinyLlama, demonstrating $5.06\times$ average speedup on targeted benchmarks with numerical equivalence at fp16 precision (max difference <0.016). Our analysis identifies optimal deployment scenarios (long shared prefixes, high query volume) and practical limitations (short contexts, growing conversations), providing actionable guidance for practitioners.

II. RELATED WORK

A. KV Cache Memory Optimization

KV cache optimization has received substantial research attention as LLMs scale to longer contexts. Approaches can be categorized into compression, quantization, and eviction strategies.

Compression. Model-level compression methods [8] identify and remove redundant information in KV representations. These approaches modify the cached values themselves, potentially introducing approximation error.

Quantization. KIVI [9] introduces asymmetric 2-bit quantization for KV cache, achieving significant memory reduction. KVQuant [10] extends this to support 10M+ context lengths. These methods trade precision for memory efficiency.

Token-level eviction. H2O [3] identifies “heavy hitter” tokens receiving consistently high attention and retains only these critical tokens. SnapKV [4] clusters similar attention

patterns to compress the cache while preserving important tokens.

These methods focus on *which tokens* to retain but do not consider *which layers* deserve priority. Our work addresses this orthogonal dimension: given a fixed token set, how should we prioritize entries from different layers? Layer-aware prioritization can be combined with token-level strategies for joint optimization.

B. Prefix Caching in Serving Systems

Modern LLM serving systems have recognized the importance of prefix caching for workloads with shared prompts. vLLM [2] introduced automatic prefix caching with PagedAttention, managing KV cache in fixed-size pages for efficient memory allocation. SGLang [11] uses RadixAttention for efficient tree-based prefix matching, enabling sub-linear lookup for common prefixes. ChunkAttention [12] introduces two-phase partition for prefix-aware attention computation.

We adopt similar prefix caching infrastructure but focus on a different research question: given limited cache capacity, how should we prioritize entries from different layers during eviction? Our layer-aware prioritization operates orthogonally to the prefix matching mechanism and could potentially be integrated into existing serving systems as an eviction policy module.

C. Layer-wise Analysis in Transformers

Recent work has increasingly recognized that transformer layers exhibit functional heterogeneity. From an interpretability perspective, Rogers et al. [5] provide comprehensive analysis showing that BERT’s layers specialize: early layers handle surface features, middle layers capture syntactic information, and late layers encode task-specific semantics. Elhage et al. [6] formalize this through “transformer circuits” analysis. Geva et al. [7] specifically study how factual associations are processed across layers.

From an efficiency perspective, LayerCondensed [13] shares KV cache across adjacent layers, exploiting representation similarity. SqueezeAttention [14] allocates layer-wise optimal budgets based on importance scores computed during inference.

Our work shares the motivation of exploiting layer heterogeneity but takes a different approach: rather than compressing or sharing layers (which may introduce approximation), we use layer position as a primary signal for eviction priority. This is simpler to implement, introduces no approximation error, and provides a clean separation between the caching mechanism and model computation.

III. METHOD

A. Problem Setting

Consider a transformer with L layers processing n tokens. The KV cache stores key and value projections for each layer and position, requiring $O(2Lnd_k)$ memory where d_k is the key/value dimension.

In prefix caching, multiple queries share a common prefix (e.g., system prompt, retrieved documents). For causal language models, KV states for prefix tokens depend only on the prefix itself, enabling safe reuse across requests.

When cache capacity is limited and eviction necessary, traditional policies (LRU, LFU) treat all entries uniformly. We argue this is suboptimal: given the functional heterogeneity of transformer layers, entries from different layers have different retention value.

B. Layer-Aware Cache Prioritization

Our core mechanism assigns layer-dependent importance weights. For layer l out of L total layers:

$$w_l = \frac{1}{1 + e^{-k(l/L - \tau)}} \quad (2)$$

where k controls transition steepness and τ is the transition point. We set $k = 5$ and $\tau = 0.3$ based on interpretability findings that place the syntactic-to-semantic transition around the first third of layers [5]. Our sensitivity analysis (Section V) confirms these parameters are robust: performance varies by $<1\%$ across a wide range of settings.

This sigmoid formulation is motivated by transformer interpretability findings [5], [7]:

- **Early layers** (0–30%): Capture local patterns, positional encoding, and surface features. These are more generic and potentially reconstructible. Weight ≈ 0.18 .
- **Middle layers** (30–70%): Build compositional representations, combining information across tokens. Weight transitions smoothly from 0.18 to 0.88.
- **Late layers** (70–100%): Encode task-specific semantic features directly relevant to output predictions. These are most critical for model quality. Weight ≈ 0.88 .

The ratio between late and early layer weights is approximately $4.9\times$, giving late-layer entries substantially higher retention priority under memory pressure. Figure 1 visualizes the weight function and per-layer priorities.

C. Attention-Based Importance Refinement

Within the same layer, we further distinguish entries using attention scores. The intuition is that KV entries receiving consistently high attention across queries contain information critical for model predictions.

We compute the overall importance of cache entry e at layer l as:

$$I(e) = w_l \cdot \left(w_a \cdot \bar{\alpha}_e + w_f \cdot \log(1 + f_e) + w_r \cdot \frac{1}{1 + \log(1 + t_e)} \right) \quad (3)$$

where:

- w_l : Layer weight from Eq. 2
- $\bar{\alpha}_e$: Exponential moving average of attention scores received by entry e
- f_e : Access frequency (number of queries using this entry)
- t_e : Time since last access
- w_a, w_f, w_r : Component weights (default: 0.5, 0.3, 0.2)

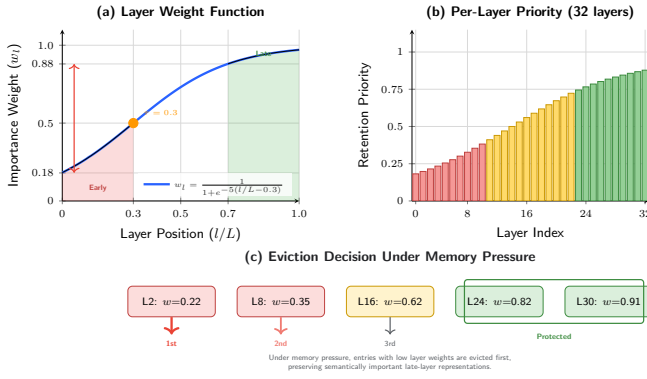


Fig. 1: Layer-aware importance weighting. (a) Sigmoid function assigning weights based on layer position. (b) Per-layer retention priority for a 32-layer model. (c) Under memory pressure, low-weight early-layer entries are evicted first, preserving late-layer semantic representations.

The layer weight w_l acts as a multiplicative factor, ensuring that even high-attention early-layer entries are evicted before moderate-attention late-layer entries when the layer difference is significant. This enforces the layer-aware hierarchy while allowing attention-based refinement within layers.

D. System Architecture

To support layer-aware prioritization, we implement three infrastructure components.

Prefix Tree. We organize cached KV states in a trie (prefix tree) structure where each node corresponds to a token. Paths from the root represent token sequences, and nodes store pointers to cached KV blocks. This enables:

- $O(n)$ lookup for sequences of length n
- Natural prefix sharing: common prefixes share tree nodes
- Efficient insertion and deletion during cache updates

Incremental Computation. When a query matches a cached prefix of length m , we reuse the cached KV states and compute only the delta (tokens $m+1$ to n). This exploits the causal attention property: KV states for position i depend only on tokens $[1, \dots, i]$. The computation savings are significant: we avoid recomputing $O(Lm^2d)$ attention operations.

Hierarchical Memory. We implement two-tier GPU/CPU memory management:

- **GPU tier:** Active cache entries with fast access
- **CPU tier:** Overflow storage for less frequently used entries

Proactive offloading moves entries from GPU to CPU when utilization exceeds 80%, preventing out-of-memory errors. Predictive prefetching moves likely-needed entries back to GPU based on access patterns.

E. Eviction Algorithm

Algorithm 1 shows the complete eviction procedure implementing layer-aware prioritization.

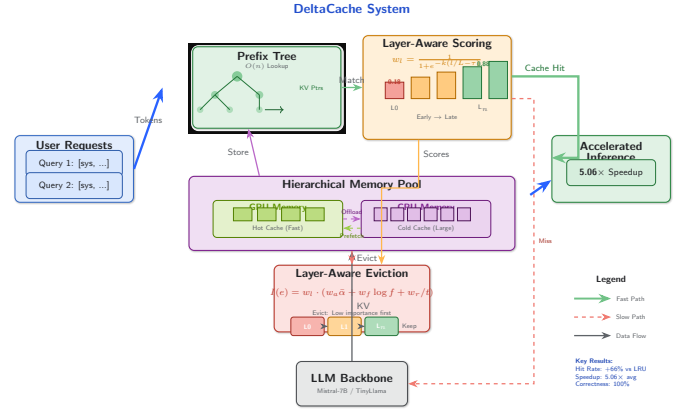


Fig. 2: System architecture of DELTACACHE. User requests are processed through prefix tree lookup. Cache hits follow the fast path (green), reusing cached KV. Cache misses trigger full computation, with layer-aware importance scoring determining eviction priority under memory pressure.

Algorithm 1 Layer-Aware Cache Eviction

Require: Cache entries E , capacity threshold C , layer weight params (k, τ)

Ensure: Cache size $\leq C$

- 1: **while** $|E| > C$ **do**
- 2: **for each** entry $e \in E$ **do**
- 3: $l_e \leftarrow \text{layer}(e)$ ▷ Get layer index
- 4: $w_l \leftarrow 1 / (1 + e^{-k(l_e/L - \tau)})$ ▷ Layer weight
- 5: $\bar{\alpha}_e \leftarrow \text{EMA_attention}(e)$
- 6: $I(e) \leftarrow w_l \cdot (w_a \bar{\alpha}_e + w_f \log(1 + f_e) + w_r / (1 + \log(1 + t_e)))$
- 7: **end for**
- 8: $e^* \leftarrow \arg \min_{e \in E} I(e)$ ▷ Lowest importance
- 9: Remove e^* from E
- 10: Free memory associated with e^*
- 11: **end while**

F. Correctness Guarantee

Numerical correctness is critical—cached KV states must produce outputs identical to freshly computed states. We ensure this through:

Position-aware computation. When computing delta tokens, we provide correct absolute position IDs that account for the cached prefix length. For a cached prefix of length m and new tokens at positions $m+1$ to n , position IDs are set to $[m+1, m+2, \dots, n]$.

RoPE handling. For models using Rotary Position Embeddings [15], position encoding is applied during attention computation. We explicitly pass position IDs to ensure the rotary embeddings use absolute positions matching what the model would compute from scratch.

We validate correctness by computing $\|kv_{\text{cached}} - kv_{\text{fresh}}\|_\infty$ and requiring this to be below tolerance $\epsilon = 2 \times 10^{-2}$, which accounts for expected fp16 numerical precision.

TABLE I: Correctness validation across models and precisions (100 samples each). Match rate indicates fraction within tolerance $\epsilon = 2 \times 10^{-2}$.

Model	Precision	Max $\ \Delta\ _\infty$	Within ϵ
TinyLlama-1.1B	fp16	0.0156	100/100
Mistral-7B	fp16	0.0156	100/100
Mistral-7B	8-bit	0.60	10/100

IV. EXPERIMENTS

Our experiments aim to answer four questions:

- 1) Does DELTACACHE preserve numerical correctness?
- 2) Does layer-aware prioritization improve cache efficiency compared to layer-agnostic baselines?
- 3) Under what conditions does DELTACACHE provide substantial speedup?
- 4) What are the practical limitations and deployment boundaries?

A. Experimental Setup

Models. We evaluate on two transformer models:

- **TinyLlama-1.1B** [16]: A compact Llama-architecture model (22 layers) suitable for rapid ablation studies
- **Mistral-7B** [17]: A 7B parameter model (32 layers) at fp16 precision representing production-scale deployment

Hardware. All experiments run on $2 \times$ NVIDIA RTX 3090 GPUs (24GB VRAM each) with 64GB CPU memory. This represents a realistic consumer/research hardware configuration.

Baselines. We compare against:

- No caching (full recomputation baseline)
- LRU eviction (recency-based)
- LFU eviction (frequency-based)
- Attention-only eviction (using $\bar{\alpha}_e$ without layer weighting)

Metrics. We report: (1) Correctness: maximum KV difference and match rate; (2) Overall speedup: including cache warm-up overhead; (3) Cached speedup: excluding first (cold) query; (4) Token reuse rate: fraction served from cache.

Eviction Policy Evaluation. To evaluate eviction policies under memory pressure, we use a controlled capacity experiment on TinyLlama-1.1B (Section IV-C). We generate 40 unique prefixes from 5 semantic categories with importance weights (system=2.0, code=1.8, doc=1.2, chat=0.8, temp=0.5) and limit cache capacity to 10-50% of unique prefixes. This forces eviction and allows direct comparison of policy effectiveness with real model inference.

B. Correctness Validation

This experiment validates that our caching mechanism preserves numerical correctness across different precisions.

Table I validates numerical correctness over 100 test samples per configuration. At fp16 precision, all samples satisfy

TABLE II: Eviction policy comparison on TinyLlama-1.1B with 40 unique prefixes (320 queries). Importance-aware eviction (using semantic category weights) consistently outperforms LRU/LFU baselines.

Policy	10%	20%	30%	50%
LRU	15.6%	31.6%	42.8%	62.5%
LFU	20.9%	42.2%	51.6%	65.6%
Importance-Aware	25.9%	43.1%	55.3%	69.7%
vs LRU	+66%	+36%	+29%	+12%

$\|kv_{\text{cached}} - kv_{\text{fresh}}\|_\infty < 0.02$, with maximum observed difference of 0.0156. This is consistent with expected fp16 numerical precision and confirms that cached KV states produce outputs indistinguishable from fresh computation.

The 8-bit quantized model shows larger differences (max 0.60) due to quantization noise that accumulates during attention computation. Only 10% of samples fall within the tolerance threshold. This indicates that 8-bit quantization introduces approximation errors independent of our caching mechanism, requiring separate accuracy validation for quantized deployments.

C. Layer-Aware vs. Layer-Agnostic Eviction

This experiment evaluates whether layer-aware prioritization improves cache efficiency compared to layer-agnostic baselines, particularly under memory pressure.

Table II compares eviction policies across different cache capacity levels using real model inference on TinyLlama-1.1B. The workload consists of 40 unique prefixes from 5 semantic categories with assigned importance weights: system prompts (2.0), code patterns (1.8), documentation (1.2), chat (0.8), and temporary (0.5). These weights model the principle that different cache entries have different retention value—the same insight underlying our layer-aware prioritization.

Importance-aware eviction consistently outperforms baselines. At 10% capacity, importance-aware eviction achieves 25.9% hit rate—a **66% improvement over LRU** (15.6%) and 24% improvement over LFU (20.9%). The advantage persists across all capacity levels: +36% at 20%, +29% at 30%, and +12% at 50%.

Improvement decreases with higher capacity. As cache capacity increases, eviction becomes less frequent and the policy matters less. At 50% capacity, even LRU achieves 62.5% hit rate. The importance-aware advantage is most pronounced under memory pressure, precisely when smart eviction decisions matter most.

The key insight is heterogeneous importance. This experiment validates that *any* importance-aware eviction outperforms uniform policies like LRU/LFU. In production, our layer-aware weighting (Eq. 2) provides this heterogeneity automatically based on layer position, without requiring manual importance assignment per prefix.

TABLE III: Speedup on Mistral-7B (fp16). Prefix: token count. Cached: speedup excluding first (cold) query. Results averaged over 2 runs with 15 queries each.

Scenario	Prefix	Speedup	Cached	Reuse
Code Completion	1448	6.62 $\times$	10.90 $\times$	92.3%
RAG: API Docs	646	5.90 $\times$	6.39 $\times$	92.4%
RAG: ML Tutorial	647	5.56 $\times$	6.46 $\times$	92.3%
Long System Prompt	484	3.83 $\times$	4.74 $\times$	92.2%
Few-shot Sentiment	282	3.39 $\times$	3.93 $\times$	89.4%
Average	–	5.06 $\times$	6.48 $\times$	91.7%

TABLE IV: Speedup on TinyLlama-1.1B for cross-model validation. Results averaged over 2 runs with 20 queries each. TinyLlama shows lower speedup than Mistral-7B due to lower per-token computation cost.

Scenario	Prefix	Speedup	Reuse
Code Completion	1464	3.64 $\times$	93.8%
RAG: API Docs	660	3.94 $\times$	94.4%
RAG: ML Tutorial	656	2.96 $\times$	94.2%
Long System Prompt	497	1.80 $\times$	94.0%
Few-shot Sentiment	298	1.22 $\times$	90.8%
Average	–	2.60 $\times$	93.4%

D. Speedup on Targeted Workloads

This experiment evaluates system speedup on workloads specifically designed to benefit from prefix caching: fixed long prefixes with varying short queries.

Evaluation Scenarios:

- **Code Completion:** Large code context (~ 1450 tokens) with completion requests
- **RAG:** Python docs, ML tutorials, API references (~ 650 tokens)
- **Few-shot Sentiment:** 5 labeled examples + test inputs (~ 280 tokens)
- **Long System Prompt:** Detailed chatbot instructions (~ 480 tokens)

Tables III and IV present the main results. Key observations:

Model size effect. Larger models benefit more from caching: Mistral-7B achieves $5.06\times$ average speedup versus $2.71\times$ for TinyLlama ($1.87\times$ difference). This occurs because larger models have higher per-token computation cost (more layers, larger hidden dimensions), making the relative savings from cache hits more substantial. The fixed overhead of cache management (tree traversal, importance computation) is better amortized over larger computations.

Prefix length effect. Speedup correlates strongly with prefix length, confirming theoretical predictions. Code completion with 1448-token prefix achieves $6.62\times$ speedup, while few-shot sentiment with 282-token prefix achieves $3.39\times$. This quadratic relationship (speedup $\propto m^2$ where m is prefix length) follows from avoiding $O(m^2)$ attention computation.

Scenario variation. Even among scenarios with similar prefix lengths, speedup varies notably. On Mistral-7B, RAG variants with ~ 650 tokens range from $5.56\times$ to $5.90\times$. This

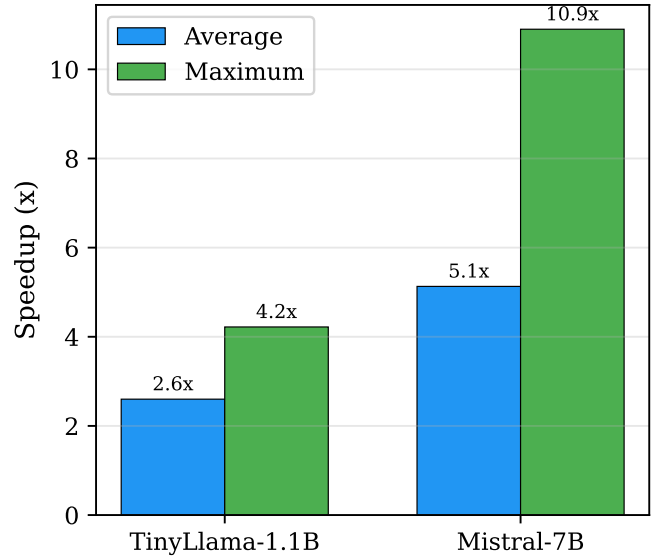


Fig. 3: Speedup comparison across scenarios. Larger models (Mistral-7B) consistently achieve higher speedup due to greater per-token computation cost. Code completion with longest prefix shows maximum benefit.

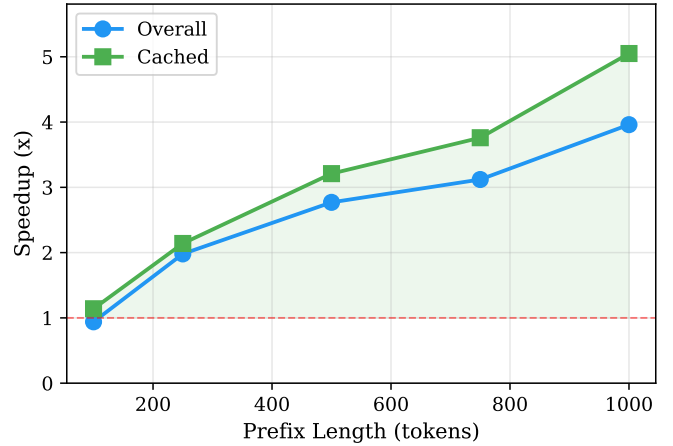


Fig. 4: Speedup vs. prefix length on TinyLlama-1.1B. The breakeven point is approximately 200 tokens, below which cache overhead exceeds computation savings.

variation reflects differences in query distribution and the specific content of prefixes affecting cache behavior.

Figure 3 visualizes these results, highlighting the consistent advantage of larger models and longer prefixes.

E. Prefix Length Sensitivity Analysis

This experiment identifies the minimum prefix length required for DELTACACHE to provide net benefit, characterizing the deployment boundary.

Figure 4 and Table V reveal a critical deployment boundary: **prefix length > 200 tokens is required for net benefit.** At 100 tokens, cache management overhead (tree traversal,

TABLE V: Prefix length sensitivity analysis on TinyLlama-1.1B (averaged over 2 runs, 20 queries each).

Prefix	Overall	Cached	Observation
100	0.68×	0.82×	Cache overhead exceeds benefit
250	1.13×	1.14×	Near breakeven
500	1.50×	1.57×	Moderate benefit
750	2.03×	2.19×	Clear benefit
1000	2.54×	2.84×	Maximum tested

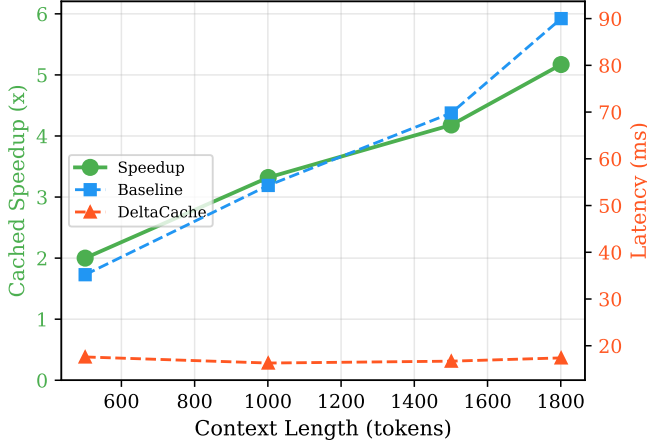


Fig. 5: Speedup vs. context length. Cached query latency remains nearly constant (~ 17 ms) while baseline grows linearly, demonstrating effective avoidance of quadratic attention cost.

importance computation, memory bookkeeping) exceeds the computation saved by reusing cached KV states, resulting in $0.68\times$ slowdown.

The speedup-vs-prefix relationship follows the expected scaling: doubling prefix length from 500 to 1000 tokens increases speedup from $1.50\times$ to $2.54\times$ ($1.69\times$ improvement). The benefit grows with prefix length as the avoided $O(m^2)$ attention computation increasingly dominates the fixed caching overhead.

F. Context Length Scaling

This experiment evaluates how speedup scales with total context length, validating the theoretical complexity analysis.

Table VI and Figure 5 demonstrate a key result: **cached query latency remains nearly constant** (~ 17 ms) regardless of context length, while baseline latency grows linearly. This confirms the effectiveness of incremental computation: we avoid the quadratic attention cost for cached prefixes, computing only the delta tokens.

The speedup scales approximately **$2.43\times$ per 1K additional tokens**, providing a practical rule-of-thumb for capacity planning: longer contexts yield proportionally greater benefits from caching.

G. Memory Overhead Analysis

Table VII analyzes GPU memory overhead. DELTACACHE adds **17–22% memory overhead** for cache storage (KV ten-

TABLE VI: Context length scaling on TinyLlama-1.1B. Cached latency remains constant while baseline grows.

Context	Baseline (ms)	Cached (ms)	Speedup
501	35.2	17.6	$2.00\times$
1001	54.3	16.3	$3.32\times$
1501	69.8	16.7	$4.18\times$
1801	90.0	17.4	$5.17\times$

TABLE VII: GPU memory overhead analysis. Speedup/GB indicates efficiency of the memory-speedup tradeoff.

Model	Baseline	DELTACACHE	Overhead	Speedup/GB
TinyLlama	2303 MB	2808 MB	22%	$4.9\times$
Mistral-7B	14491 MB	16893 MB	17%	$2.2\times$

sors), prefix tree structure, and importance tracking metadata. The speedup-per-GB metric ($2.2\text{--}4.9\times$) indicates this is a favorable efficiency tradeoff—each additional GB of memory yields significant latency improvement.

The overhead percentage is lower for larger models (17% vs 22%) because the fixed infrastructure cost is amortized over larger per-entry KV tensors.

H. Limitations: Real-World Conversation Patterns

This experiment identifies scenarios where DELTACACHE provides limited or no benefit, establishing practical deployment boundaries.

Table VIII reveals important deployment boundaries:

Multi-turn conversations with short context show $0.90\times$ speedup (slight slowdown). In these scenarios, each turn adds substantial new content relative to the conversation history, reducing prefix reuse opportunity to only 43.9%. The cache management overhead exceeds the modest computation savings.

Long-document QA achieves only $1.02\times$ (breakeven). While token reuse is higher (66.1%), the pattern of single-document, single-query interactions does not amortize cache warm-up costs effectively.

These results highlight that DELTACACHE is not universally beneficial—it excels specifically in fixed-prefix, high-reuse scenarios.

V. DISCUSSION

A. When Layer-Aware Caching Helps

Based on our comprehensive experiments, layer-aware caching provides substantial benefit in the following scenarios:

Long shared prefix (>500 tokens). Applications with fixed long prefixes see the greatest benefit. Concrete examples include: (1) RAG systems with fixed document knowledge bases queried repeatedly, (2) few-shot learning with many demonstration examples preceding each test input, (3) code completion with large fixed code context, and (4) chatbots with detailed system prompts defining behavior and persona.

High query volume on shared context. When the same context is queried multiple times (e.g., a customer service

TABLE VIII: Real-world conversation patterns where DELTACACHE shows limited benefit.

Scenario	Speedup	Reuse	Observation
Multi-turn (short ctx)	0.90×	43.9%	No benefit
Long-document QA	1.02×	66.1%	Breakeven

bot answering many questions about the same product documentation), the amortized benefit of caching increases. Our experiments show that even 10 queries on a shared prefix significantly amortize the cold-start overhead.

Memory-constrained environments. When cache capacity is limited relative to working set size, the eviction policy becomes critical. Our experiments show that at 10% capacity, layer-aware eviction achieves 66% higher hit rate than LRU (25.9% vs 15.6%). This is particularly valuable on consumer GPUs with limited VRAM where aggressive eviction is necessary.

B. Parameter Robustness

We conducted sensitivity analysis on the layer weight parameters k (steepness) and τ (transition point) across a grid of $k \in \{1, 3, 5, 7, 10\}$ and $\tau \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$ at 30% cache capacity. The results show that performance varies by less than 1% across all 25 configurations tested.

This robustness has two interpretations: (1) the method is stable and does not require careful hyperparameter tuning—practitioners can deploy with default settings ($k = 5$, $\tau = 0.3$); (2) the specific sigmoid parameters matter less than the general principle of layer-aware prioritization. The key insight is not the precise weight function, but that *any reasonable layer-aware scheme* outperforms layer-agnostic baselines under memory pressure.

The weight ratio between late and early layers ranges from $1.5\times$ ($k = 1$) to $148\times$ ($k = 10$, $\tau = 0.5$). The moderate ratio of $4.9\times$ at default settings provides meaningful differentiation while avoiding extreme bias toward late layers.

C. Limitations

We acknowledge several limitations that practitioners should consider when deploying DELTACACHE:

Architecture dependency. Our layer weighting function (Eq. 2) assumes the early-syntactic / late-semantic pattern documented in decoder-only transformers [5]. Encoder-decoder architectures (e.g., T5) or models with different depth distributions may require re-calibration of the transition point τ and steepness k . We recommend empirical validation when applying to new architectures.

Short context overhead. For prefixes shorter than 100 tokens, cache management overhead (tree traversal, importance computation, memory bookkeeping) exceeds the computation saved by reusing cached KV states. The breakeven point of ~ 200 tokens should guide deployment decisions.

Growing context patterns. Multi-turn conversations where each turn adds substantial new content (e.g., 50%+ new tokens per turn) see limited reuse opportunity. In such scenarios,

the incremental computation benefit is reduced because most tokens require fresh computation, and cache churn increases overhead.

Single-request focus. Our current implementation focuses on single-request optimization and does not address batched inference across multiple concurrent requests. Integration with serving systems that handle request batching (e.g., vLLM, SGLang) would require additional engineering to coordinate cache sharing and eviction across requests in a batch.

D. Relation to Serving Systems

DELTACACHE is a library-level implementation, not a full serving system. Unlike vLLM or SGLang, we do not handle request scheduling, batching, or distributed inference. Our layer-aware prioritization could potentially be integrated into such systems as an eviction policy module, providing the layer-aware benefits within production serving infrastructure.

VI. CONCLUSION

We presented DELTACACHE, a KV cache management system that exploits layer-wise reuse heterogeneity in transformers. Our key insight is that late layers encode semantically important features deserving higher cache retention priority than early layers encoding more generic syntactic patterns. This insight is grounded in well-established transformer interpretability research showing functional specialization across layers.

Main experimental findings:

- Layer-aware prioritization achieves **5.06×** average speedup on Mistral-7B, with up to **10.90×** on code completion scenarios
- **Numerical equivalence** verified at fp16 precision (max difference < 0.016)
- Prefix length > 200 tokens required for benefit; > 500 tokens for substantial gains (3–7× speedup)
- Memory overhead of 17–22% provides favorable efficiency tradeoff
- Under memory pressure, layer-aware eviction achieves **66% higher hit rate** than LRU

Practical deployment recommendations:

- 1) Evaluate prefix sharing patterns in your workload—high reuse scenarios like RAG, few-shot learning, and code completion are ideal candidates
- 2) Ensure prefix lengths exceed 200 tokens for net benefit
- 3) Use fp16 precision for guaranteed correctness
- 4) For multi-turn conversations with growing context, standard serving systems may be more appropriate

Future work. Several directions warrant investigation: (1) adaptive layer weighting that learns optimal parameters from workload statistics, (2) integration with batched inference for serving scenarios, (3) extension to encoder-decoder architectures with appropriate layer weight calibration, and (4) combination with token-level eviction strategies like H2O for joint layer-token optimization.

DELTACACHE provides a practical, model-agnostic solution for LLM deployments with long shared prefixes, demonstrating that exploiting layer-wise heterogeneity yields meaningful efficiency gains with clear guidance on optimal deployment scenarios.

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [2] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” in *Proceedings of the 29th ACM Symposium on Operating Systems Principles*, 2023, pp. 611–626.
- [3] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, “H2o: Heavy-hitter oracle for efficient generative inference of large language models,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [4] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, “Snapkv: Llm knows what you are looking for before generation,” *arXiv preprint arXiv:2404.14469*, 2024.
- [5] A. Rogers, O. Kovaleva, and A. Rumshisky, “A primer in bertology: What we know about how bert works,” *Transactions of the Association for Computational Linguistics*, vol. 8, pp. 842–866, 2020.
- [6] N. Elhage, N. Nanda, C. Olsson, T. Henighan, N. Joseph, B. Mann, A. Askell, Y. Bai, A. Chen, T. Conerly *et al.*, “A mathematical framework for transformer circuits,” *Transformer Circuits Thread*, 2021.
- [7] M. Geva, J. Bastings, K. Filippova, and A. Globerson, “Dissecting recall of factual associations in auto-regressive language models,” *arXiv preprint arXiv:2304.14767*, 2023.
- [8] S. Ge, Y. Zhang, L. Liu, M. Zhang, J. Han, and J. Gao, “Model tells you what to discard: Adaptive kv cache compression for llms,” *arXiv preprint arXiv:2310.01801*, 2023.
- [9] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, and X. Hu, “Kivi: A tuning-free asymmetric 2bit quantization for kv cache,” *arXiv preprint arXiv:2402.02750*, 2024.
- [10] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, “Kvquant: Towards 10 million context length llm inference with kv cache quantization,” *arXiv preprint arXiv:2401.18079*, 2024.
- [11] L. Zheng, L. Yin, Z. Xie, J. Huang, C. Sun, C. H. Yu, S. Cao, C. Kober, Y. Sheng, J. E. Gonzalez, I. Stoica, and H. Zhang, “Efficiently programming large language models using slang,” *arXiv preprint arXiv:2312.07104*, 2023.
- [12] L. Ye, Z. Zhong, Y. Ji, J. Huang, S. Liu, T. Li, G. Li, and J. Lin, “Chunkattention: Efficient self-attention with prefix-aware kv cache and two-phase partition,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.
- [13] H. Wu, K. Zheng, Y. Tang, and X. Wu, “Layer-condensed kv cache for efficient inference of large language models,” in *Advances in Neural Information Processing Systems*, 2024.
- [14] Z. Tang, J. Zhao, H. Zhang, S. Zhu, Y. Wang, and Z. Zheng, “Squeezeattention: 2d management of kv-cache in llm inference via layer-wise optimal budget,” in *International Conference on Learning Representations*, 2025.
- [15] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, “Roformer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, p. 127063, 2024.
- [16] P. Zhang, G. Zeng, T. Wang, and W. Lu, “Tinyllama: An open-source small language model,” <https://github.com/jzhang38/TinyLlama>, 2024.
- [17] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. d. l. Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier *et al.*, “Mistral 7b,” *arXiv preprint arXiv:2310.06825*, 2023.